University of Glasgow

School of Computing Science

Blockchain-Based Electronic Voting

Literature Review and Research Methodology

Mathematical modelling and experimental evaluation of
bribery, Sybil and ballot-stuffing attacks

MSc Computing Science

Dissertation — extract

Author, supervisor and submission date to be completed by the candidate.

This document contains two chapters of the dissertation — the literature review and the research design and methodology — together with a short summary of the formal model they refer to. The results themselves are reported in the full dissertation. References to chapters not reproduced here are named rather than numbered. Experimental figures quoted in the methodology are injected from the accompanying simulation output (seed 42, 2,500 paired trials) and are reproducible bit-for-bit. No real election system, voter record or personal datum is used anywhere in this work.

Contents

1 Preliminaries: the model in brief **2**

1.1 Threat model . . . 2

1.2 The equations referred to below . . . 2

2 Literature Review **4**

2.1 Scope and method of the review . . . 4

2.2 Electronic voting before blockchain . . . 4

2.3 Blockchain-based voting proposals . . . 5

2.4 The critical response: what happens on deployment . . . 5

2.5 Bribery and vote buying as a formal problem . . . 6

2.6 Sybil attacks and identity . . . 7

2.7 Ballot stuffing and statistical election forensics . . . 7

2.8 Probabilistic and game-theoretic security analysis . . . 8

2.9 Synthesis and research gap . . . 8

3 Research Design and Methodology **10**

3.1 Research strategy: why simulation . . . 10

3.2 A two-layer architecture, and why both layers are necessary . . . 10

3.3 Variables and operationalisation . . . 11

3.4 What the ledger contributes, and what it does not . . . 12

3.5 Parameter selection and its justification . . . 12

3.6 Paired randomised design . . . 13

3.7 Triangulation of three computational methods . . . 13

3.8 Threshold solver design . . . 14

3.9 Coalition search design . . . 15

3.10 Validation and verification . . . 15

3.11 Reproducibility protocol . . . 15

3.12 Ethical considerations . . . 16

References **17**

1 Preliminaries: the model in brief

This section states just enough of the formal model for the two chapters that follow to be read on their own. It is a summary of the dissertation's model chapter, not a substitute for it: the derivations, edge-case conventions and correctness arguments are given there.

1.1 Threat model

A single attacker attempts to change the outcome of a two-candidate election with N enrolled voters. The *manipulation threshold* m is the number of additional fraudulent votes needed to overturn the honest margin,

$$m = (\text{opponent votes} - \text{target votes}) + 1, \tag{1}$$

and the attack succeeds precisely when at least m fraudulent votes survive every countermeasure and reach the final count. Three vectors are available: **bribery** of genuine voters, constrained by a budget B at a reward R per voter and detected with probability q; **Sybil registration** of S forged identities, each accepted with probability q_s and detected with probability p_d; and **ballot stuffing** of A fraudulent ballots, each accepted with probability q_b and detected with probability p_b.

1.2 The equations referred to below

A voter offered a bribe faces a decision under risk, gaining R if undetected and paying a penalty P if not:

$$U_v = (1-q)R - qP, \tag{2}$$

and a risk-neutral voter accepts when $U_v > 0$. This acts as a *gate*: where bribery is not incentive-compatible, no bribe is accepted and the attack contributes nothing at any budget.

Attempts within a vector are independent Bernoulli trials, so the count of surviving fraudulent votes is binomial and the success probability is its survival function:

$$\mathbb{P}(W \geq m) = \sum_{k=m}^{n} \binom{n}{k} p^k (1-p)^{n-k}, \qquad \mathbb{E}[W] = \mu = np. \tag{3}$$

A forged identity or an injected ballot must clear two independent gates to be counted — acceptance and non-detection — so the surviving count is

$$W \sim \text{Binomial}\big(n,\ q_{\text{accept}}(1 - p_{\text{detect}})\big). \tag{4}$$

Finally, the attacker's expected utility from T bribery attempts, given a payoff V for flipping the outcome and a penalty D if the attack is noticed at all, is

$$U(T) = \mathbb{P}(X \geq m)V \ - \ TR \ - \ \big[1 - (1-q)^T\big]D. \tag{5}$$

Throughout, q denotes the *detection* probability, so a bribed vote survives with probability $1 - q$.

2 Literature Review

2.1 Scope and method of the review

This review covers four bodies of work that the dissertation draws together, and it is organised by the question each body answers rather than chronologically.

1. **What has been built, and what is claimed for it?** Electronic and blockchain-based voting systems, from cryptographic verifiable-voting schemes to ledger-backed proposals (Sections 2.2–2.3).
2. **What breaks when it is deployed?** The empirical security literature: analyses of fielded internet-voting systems and the critical response to blockchain voting specifically (Section 2.4).
3. **How is manipulation modelled formally?** Bribery as a computational problem, coercion and receipt-freeness, Sybil identity attacks, and the statistical forensics of fraudulent tallies (Sections 2.5–2.7).
4. **What tools exist for reasoning about a rational, resource-bounded attacker?** Concentration inequalities and game-theoretic security analysis (Section 2.8).

Sources were located by targeted search of the primary security venues (USENIX Security, ACM CCS, Financial Cryptography, IEEE S&P and their workshops), the artificial-intelligence and economics literature for the manipulation-theory strand, and two recent systematic reviews of blockchain voting [16, 18] used as entry points into the applied literature. Priority was given to peer-reviewed primary sources; where a widely-cited result exists only as a technical report or research blog post, as with the Dark DAO construction [6], this is stated explicitly rather than disguised by citation formatting.

Section 2.9 closes the review by identifying what the four bodies of work leave unaddressed, which is the gap this dissertation occupies.

2.2 Electronic voting before blockchain

The security requirements for an electronic election were substantially settled before distributed ledgers entered the conversation, and it is worth restating them because blockchain proposals are frequently evaluated against a narrower list.

Verifiability. A voter should be able to confirm that their ballot was recorded as cast, and any observer should be able to confirm that the recorded ballots were tallied correctly. Adida's Helios [1] is the canonical open-audit implementation: it publishes encrypted ballots to a public bulletin board and proves the tally correct with homomorphic encryption and zero-knowledge proofs, so that correctness rests on mathematics rather than on trusting the election authority. Helios matters to the present work as a counterexample to a common framing: it delivers public verifiability with no ledger, no chain and no consensus mechanism, which shows that the verifiability argument for blockchain voting is not an argument for blockchain specifically.

Ballot secrecy and its stronger forms. Secrecy alone is insufficient if a voter can *prove* how they voted, because a provable vote is a sellable vote. Benaloh and Tuinstra [3] formalised *receipt-freeness*: the

requirement that a voter cannot construct a convincing proof of their choice even if they want to. Juels, Catalano and Jakobsson [10] strengthened this to *coercion-resistance*, covering an adversary who can interact with the voter during voting, demand credentials, or require abstention; their scheme lets a coerced voter surrender a fake credential indistinguishable from a real one.

This distinction sets the boundary of the present work. Bribery, as modelled here, presupposes that the briber can verify compliance well enough for the transaction to be worth making. A system that is genuinely receipt-free undermines that presupposition at the root, which is a different and stronger defence than the detection-based one modelled in this dissertation. The *Limitations* chapter of the full dissertation returns to this.

2.3 Blockchain-based voting proposals

Nakamoto's chained proof-of-work construction [15] supplied a mechanism with obvious surface appeal for elections: an append-only, hash-linked, replicated log in which retrospective edits invalidate every subsequent hash and are therefore detectable by anyone holding a copy.

The most rigorous academic instantiation is McCorry, Shahandashti and Hao's Open Vote Network [13], a boardroom-voting protocol implemented as an Ethereum smart contract, which achieves maximum voter privacy in the sense that a vote can only be revealed by full collusion of all remaining participants, and which is self-tallying: no trusted authority computes the result. It is a genuine cryptographic contribution rather than a ledger with ballots written into it. Its scope, however, is explicitly the boardroom — tens of voters, all known to one another — and its cost and collusion properties do not transfer to a public election.

The wider applied literature is large and, by the assessment of its own systematic reviewers, uneven. Ohize et al. [16] survey architectures and trends across the field and identify scalability, post-quantum security, voter authentication, regulatory frameworks and cross-chain interoperability as the persistent open problems. Rahul, Gulia and Gill [18] review 55 journal articles and reach a similar conclusion, noting that security, transparency and trust remain unresolved worldwide rather than settled by any existing proposal. Two observations from these reviews shape the present work. First, *voter authentication* is repeatedly named as an open problem, which is precisely the Sybil vector modelled here. Second, the surveyed proposals are overwhelmingly evaluated on functionality, throughput and cryptographic properties rather than on quantified resistance to a budgeted attacker — the gap this dissertation addresses.

2.4 The critical response: what happens on deployment

Against the proposal literature stands a smaller and considerably more sobering body of empirical analysis.

Springall et al. [20] examined Estonia's I-voting system, the most established national internet-voting deployment, by observing operations directly and reconstructing the system in the laboratory. They reported serious procedural lapses and architectural weaknesses sufficient, in their assessment, for a state-level attacker to alter results undetectably, and recommended discontinuing the system. The finding relevant here is not the specific vulnerabilities but the location of them: the client platform and the

operational procedures surrounding the count, not the cryptography.

Specter, Koppel and Weitzner [19] reverse-engineered Voatz, the first internet-voting application used in U.S. federal elections and marketed on its use of a blockchain. They found flaws allowing an adversary to alter, stop or expose a voter's ballot, a side-channel through which a passive network observer could recover the secret ballot, and a dependence on third-party services with attendant privacy exposure; West Virginia and a Washington county withdrew the application from 2020 primaries following the analysis. The architectural point is the important one: the blockchain sat behind an application whose compromise made the ledger irrelevant, because votes were corrupted before they reached it.

Park, Specter, Narula and Rivest [17] generalise these observations into a direct argument that blockchain voting is worse than internet voting rather than better. Their case is that blockchains address a problem elections do not have — disagreement about which transactions occurred among mutually distrusting parties — while introducing new failure modes, and that the genuinely hard problems of elections, namely software independence, secret ballots, voter authentication and client-side integrity, are untouched. They further note that a blockchain vote is a *transparent* record, which is in tension with ballot secrecy rather than supportive of it.

This dissertation adopts that critique as a premise to be tested quantitatively rather than as a conclusion to be restated. The experimental design deliberately grants the ledger everything its proponents claim — integrity is verified continuously and, as the *Findings* chapter reports, was never violated in any trial — and then measures how many elections are stolen anyway.

2.5 Bribery and vote buying as a formal problem

Three distinct traditions model vote buying, and this dissertation borrows from all three.

Computational. Faliszewski, Hemaspaandra and Hemaspaandra [9] formalise bribery as a decision problem: given per-voter prices and a budget, can the briber make a preferred candidate win? They classify the complexity across voting rules, establishing that some systems make optimal bribery NP-hard — complexity as a defensive barrier. The framing is worst-case, combinatorial and deterministic: a bribed voter votes as paid.

Economic. Becker's analysis of crime as a decision under risk [2] models an offender weighing gain against the probability of detection multiplied by the penalty, and derives the policy conclusion that deterrence can be achieved either by raising detection or by raising penalties. Equation (2) in this dissertation is exactly that calculation applied to a bribed voter, and the critical-detection threshold developed in this work is the policy question in Becker's sense: how much detection is enough?

Blockchain-specific. Daian, Kell, Bentov and Juels [6] describe on-chain vote buying and the *Dark DAO*: a smart contract that pays for votes and enforces the payment trustlessly, using trusted execution to verify compliance without revealing the buyer. Their argument deserves emphasis in the present context because it inverts the usual claim. A blockchain does not merely fail to prevent vote buying; by making votes cryptographically attributable and payments programmable and escrowed, it *improves* the market for votes, removing the counterparty risk that limits bribery in traditional settings. This is a technical-report and research-blog contribution rather than a peer-reviewed paper, and is cited as such,

but it is widely referenced and directly motivates treating bribery, not tampering, as the primary threat to a ledger-based election.

The gap across all three: the computational tradition treats bribery deterministically, the economic tradition treats the individual decision without an electorate-level outcome model, and the blockchain-specific work is a mechanism proposal rather than a quantified risk model. None produces the quantity a defender needs — the probability that a given budget overturns a given margin against a given detection capability.

2.6 Sybil attacks and identity

Douceur [7] established that, absent a trusted certification authority, a single entity in a distributed system can present arbitrarily many distinct identities, and that no purely local resource test reliably distinguishes them. For an election the consequence is immediate and fatal: one-person-one-vote is the entire premise, and a Sybil attack attacks the premise rather than the mechanism.

Responses divide into those that reintroduce a trusted authority and those that attempt to establish uniqueness without one. Borge et al. [4] propose proof-of-personhood, binding digital identities to physical presence at pseudonym parties, explicitly to recover one-per-person semantics in permissionless systems. The approach illustrates both the necessity and the difficulty: physical attendance is a strong Sybil barrier and an equally strong barrier to accessibility, which is what electronic voting exists to improve.

This dissertation does not model any specific identity mechanism. It parameterises the outcome instead — a forged registration passes with probability q_s and is caught with probability p_d — which is weaker than assuming a trusted authority and is deliberately conservative. It also produces the finding, reported in the *Findings* chapter, that identity verification rather than ledger integrity is where the residual risk concentrates, consistent with voter authentication being repeatedly flagged as an open problem in the systematic reviews [16, 18].

2.7 Ballot stuffing and statistical election forensics

Ballot stuffing — injecting fraudulent ballots not corresponding to any voter — is the least cryptographically interesting of the three vectors and, historically, among the most common. The relevant literature is statistical rather than cryptographic.

Klimek, Yegorov, Hanel and Thurner [11] show that systematic irregularities leave detectable signatures in the joint distribution of turnout and vote share across polling units: incremental stuffing and extreme fraud produce characteristic distortions absent from clean elections, and the authors identify these fingerprints in several national elections. The methodological lesson carried into this dissertation is that fraud is bounded by *plausibility* as much as by opportunity. An attacker who injects ballots without limit produces a turnout figure that requires no probabilistic detection to notice. This is the justification for the capacity ceilings imposed in the coalition model (described in the methodology below), which are otherwise a modelling choice without grounding: they stand in for the anomaly detection that election forensics performs.

2.8 Probabilistic and game-theoretic security analysis

The final strand supplies the analytical machinery.

Concentration inequalities. Chernoff's bound [5], in the multiplicative form given by Mitzenmacher and Upfal [14], bounds the probability that a sum of independent Bernoulli variables deviates from its mean, and does so exponentially in the deviation. For this dissertation its value is specific: it converts an empirical statement into a provable one. A margin certified safe by the bound is safe irrespective of how many simulation runs were performed, which is what makes the certified margin defined in this work a security claim rather than an observation.

Rational attackers. Eyal and Sirer [8] demonstrated with selfish mining that a blockchain protocol assumed secure under an honest-majority model can be broken by a rational minority that deviates profitably — security depending on incentives, not only on cryptography or on hash-power share. Liu et al. [12] survey the resulting body of game-theoretic blockchain analysis, covering mining strategy, security games and incentive design.

Two points transfer directly. First, the relevant question about an attacker is what they would *choose* to do, not what they are capable of doing: the *Findings* chapter shows that a budget-exhausting attacker performs strictly worse than a rational one, so a defence tuned against maximum expenditure is tuned against the wrong opponent. Second, deterrence is a legitimate security goal distinct from prevention — making an attack unprofitable is cheaper than making it impossible, and the gap between those two thresholds is quantified in this dissertation.

2.9 Synthesis and research gap

Table 1 summarises what each strand supplies and what it leaves open.

Table 1. What the literature supplies, and what it leaves open.

Strand	Supplies	Leaves open
Blockchain voting proposals [13, 16, 18]	Architectures, cryptographic protocols, throughput evaluation	No quantified success probability against a budgeted attacker
Empirical critiques [20, 19, 17]	Evidence that deployments fail upstream of the ledger	Qualitative argument; no model of how much manipulation gets through
Bribery theory [9, 2, 6]	Complexity classification; the rational-offender calculus; on-chain vote-buying mechanisms	Deterministic or individual-level; no electorate-scale outcome distribution
Sybil and identity [7, 4]	Impossibility without trusted certification; countermeasure designs	No probabilistic model of partial identity verification
Election forensics [11]	Statistical signatures of stuffing; plausibility as a real constraint	Detection after the fact, not a design threshold beforehand
Probabilistic and game-theoretic tools [5, 14, 8, 12]	Concentration bounds; rational-deviation analysis	Not yet applied to e-voting manipulation as an integrated model

Three gaps follow, and they define the contribution of this dissertation.

Gap 1: no quantified attacker model for e-voting countermeasures. The proposal literature evaluates systems on cryptographic properties and performance; the critical literature establishes qualitatively that deployments fail. Neither produces the probability that a specific budget overturns a specific margin against a specific detection capability. The *Threat Model* and *Findings* chapters supply exactly that, over 2,500 matched randomised trials per vector.

Gap 2: probabilities are reported where thresholds are needed. A defender cannot act on a success-probability curve. The inverse question — what detection probability, what budget ceiling, what margin makes this attack non-viable — is not asked in the e-voting literature, although Becker's economics of deterrence [2] and the game-theoretic blockchain work [8, 12] both imply it should be. The *Critical Thresholds* chapter formulates seven such thresholds and solves them.

Gap 3: attack vectors are evaluated in isolation. Every source reviewed treats bribery, Sybil attacks and ballot stuffing separately. No reviewed work models an attacker allocating one budget across several vectors simultaneously. The *Coalition Attacks* chapter shows this omission is not conservative: two of the three vectors are individually infeasible in every benchmark scenario, yet their combination succeeds decisively in four of five.

3 Research Design and Methodology

3.1 Research strategy: why simulation

The research questions ask what *would* happen to an election under attack. That rules out the two obvious empirical strategies. A field experiment is unethical and illegal: it would require attempting to corrupt a real election. Observational study of historical fraud suffers from severe selection bias, because only detected fraud is observable and detected fraud is by definition the fraud that failed at the part this dissertation measures.

The strategy adopted is therefore **computational simulation with analytical validation**: construct a formal model of the attack, compute its consequences exactly where closed forms exist, simulate where they do not, and validate the two against each other. The claims that result are conditional — "under this model, with these parameters, the following follows" — and the burden of the design is to make the conditions explicit, the computation correct, and the results reproducible.

Three commitments follow, and they shape everything below.

Exactness before estimation. Where a quantity has a closed form it is computed exactly, not estimated. Attack success probability is the binomial survival function (Equation (3)), evaluated through the regularised incomplete beta function; the coalition distribution is an exact convolution (Section 3.9); the threshold solvers are exact binary searches or bisections on monotone functions. Monte Carlo is used to *validate* these exact values and to reach quantities that have no closed form, not as the primary method. This inverts the more common simulation-study design, and it is what allows the Monte Carlo error itself to be reported as a result (see the *Findings* chapter).

Falsifiability by construction. Wherever a modelling decision could flatter the conclusion, the design includes a mechanism that would expose it. The Chernoff bound is checked against the exact value in every trial, so a wrongly-implemented bound would show as a violation rather than as a plausible number. The coalition convolution is checked against a 200,000-draw Monte Carlo simulation in the test suite. Reproducibility is enforced by a test that fails if two runs at the same seed differ, and by a second test that fails if two runs at *different* seeds agree — so the first test cannot pass vacuously.

Assumptions declared and swept. Every modelling assumption is recorded in `ASSUMPTIONS.md` and cross-referenced from the module that relies on it. Where an assumption is both consequential and ungrounded — the relative costs of the three vectors, the capacity ceilings — it is not merely declared but swept, and the conclusion is reported as a function of it (see the *Findings* chapter).

3.2 A two-layer architecture, and why both layers are necessary

The implementation has two layers that answer the same questions by different means.

The **concrete layer** is a working blockchain e-voting application: SHA-256 block hashing, proof-of-work mining, chain validation, voter registration, role-based election administration, duplicate-vote prevention and result publication. Attacking it produces an actual winner change on an actual mined chain.

The **abstract layer** is the mathematical framework: exact binomial evaluation, Chernoff bounds, Monte

Carlo, the threshold solvers and the game-theoretic optimisers. It produces probabilities rather than outcomes.

Neither layer alone would support the conclusions. The abstract layer supplies statistical power — 2,500 paired trials complete in 6.0 seconds, whereas mining that many elections is infeasible — but every one of its numbers is a probability computed under a model, and a model can be wrong in ways that are invisible from inside it. The concrete layer supplies ground truth — a real tally, a real winner, a real chain whose integrity is checked after every attack — but at a sample size (300 elections) too small for the parameter sweeps.

Running both and comparing them is a validation strategy, not duplication. If the abstract model were mis-specified — if, say, the survival gates of Equation (4) did not correspond to what actually happens to a fraudulent ballot in the concrete system — the two layers would disagree in ordering or magnitude. The *Findings* chapter reports that they agree in both.

3.3 Variables and operationalisation

Table 2 states how each construct is operationalised. The distinction that matters most is between the three *independent* variables the experiment manipulates and the *control* variables that are held identical across the paired arms.

Table 2. Operationalisation of the model's variables.

Role	Variable	Operationalisation
Independent	System configuration	Two levels: baseline (weak detection and validation) and protected (blockchain-style audit, signatures, identity verification, ballot validation — modelled as higher detection and lower acceptance probabilities).
Independent	Attack vector	Three levels: bribery, Sybil registration, ballot stuffing; plus the coalition of all three in the coalition analysis.
Independent	Attacker resources	Budget B and reward R, entering through the feasibility constraint $T = \min(N, \lfloor B/R \rfloor)$.
Dependent	Attack success probability	$\mathbb{P}(W \geq m)$, where W counts fraudulent votes surviving every countermeasure. Computed exactly; also estimated by Monte Carlo and bounded by Chernoff.
Dependent	Election flip	Binary: did the winner of a concretely mined election change? Measured only in the concrete layer.
Dependent	Attacker expected utility	Equation (5): payoff weighted by flip probability, less cost, less expected penalty.
Control	Scenario structure	N, m, B, R, P, S, A — identical between the two arms of every pair by construction.
Control	Random stream	A single seeded generator; identical seed gives identical output bit-for-bit.

3.4 What the ledger contributes, and what it does not

Because the protected configuration bundles several controls together, it is worth stating precisely which component does what before any result is reported. The pipeline is ordered, and the ordering is the whole point:

fraud attempt → **acceptance / identity / signature / ballot validation** → **detection** → surviving votes only → **ledger records them**

Every fraudulent vote that is stopped in this model is stopped by the validation or detection stage. The ledger sits at the end of the pipeline and records whatever survives; it contributes integrity, immutability, auditability and tamper evidence, and it contributes nothing whatever to deciding *whether a vote should have been cast*. A blockchain cannot tell that a genuine voter was paid, that a registration was forged, or that a ballot was injected — those are determined upstream, by mechanisms that would work equally well behind a conventional append-only audit log.

This distinction is preserved in the experimental design rather than merely asserted in the prose. The concrete layer runs the chain's own validity check after every attack, so the two properties are measured separately: the ledger's integrity, and the election's correctness. The *Findings* chapter reports that these come apart completely — integrity held in every trial, and elections were flipped anyway.

Nothing in this dissertation should therefore be read as showing that "blockchain prevents bribery", or Sybil attacks, or ballot stuffing. The claim supported by the results is narrower and conditional: under the assumptions of this model and over the sampled parameter ranges, a protected architecture combining ledger-backed auditability with identity verification, signature validation, ballot validation and raised detection substantially reduces attack feasibility.

The dependent variable deserves one clarification. "Attack success" is defined throughout as reaching the manipulation threshold m, which is the number of additional fraudulent votes needed to overturn the margin (Equation (1)). It is not defined as "some fraud occurred" or "fraud was undetected". This is a deliberately demanding definition: an attack that places a hundred fraudulent votes into an election it fails to flip counts, here, as a failure.

3.5 Parameter selection and its justification

Parameter ranges (Table 3) were selected to span plausible deployment conditions rather than to be estimated from data, because for most of these quantities no defensible estimate exists. The probability that a forged voter registration is detected is not published by any electoral authority in a form that would support a point estimate. Three consequences follow, and the design accommodates all three.

Ranges, not points. Every parameter is sampled from a range on every trial, so no result depends on a single assumed value. The electorate spans 200 to 4,999 voters and the threshold m is itself randomised as 2–25% of N, so the reported means are averages over a distribution of scenarios rather than properties of one configuration.

Direction of bias stated. Where a choice could flatter either side it is made against the conclusion being

argued. Legitimate ballots always pass validation, so the model never credits the defence with catching fraud by accident. Detection is applied independently per item, which is more favourable to the attacker than correlated detection, since an audit that catches one item usually catches its batch.

Sensitivity where it matters. The coalition analysis depends on four constants with no empirical basis — two relative costs and two capacity ceilings. Rather than defend particular values, the analysis re-runs across 108 combinations of them and reports the conclusion's stability (see the *Findings* chapter).

The benchmark scenarios (S1–S5) are calibrated differently and for a different purpose. For S1–S4 the threshold m is placed close to the baseline mean μ rather than at an extreme. This is necessary for the three-method comparison to be informative: a threshold far below μ makes success trivially certain, one far above makes it trivially impossible, and the Chernoff bound is not even defined when $m \leq \mu$. S5 is the deliberate exception — a threshold far beyond what its budget can reach — and functions as a negative control: a model that reported S5 as succeeding would be wrong in an obvious way.

3.6 Paired randomised design

The core experiment is a **matched-pairs design**. Each trial draws one scenario and evaluates it twice, under both configurations, with identical N, m, B, R, P, S, A and identical random streams. Only the acceptance and detection probabilities differ.

The alternative — drawing independent samples for each arm — would confound the countermeasure effect with the difference between the scenarios happening to be drawn for each arm. Because the per-trial success probability has a standard deviation of 0.474 for bribery, scenario-to-scenario variance dominates the effect being measured, and an unpaired design would need far more trials to reach the same precision. Pairing removes that variance entirely: the reported reduction is a within-scenario effect, and every pair is its own control.

Inference. Confidence intervals are computed by bootstrap resampling of the paired differences (10,000 resamples) rather than by a normal approximation. This is not a stylistic preference. The per-trial success probabilities are strongly bimodal — most scenarios are either comfortably safe or comfortably lost — which is visible in the summary statistics: the baseline bribery distribution has a mean of 0.350 but a median of 0.0000. A distribution with that shape violates the assumptions of a t-interval, and the bootstrap makes no distributional assumption.

Trial count. 2,500 pairs per vector were run, above the 1,000 originally specified, chosen so that the bootstrap intervals on the mean reduction are narrow relative to the effect: the bribery reduction interval $[0.269, 0.304]$ has a width under 0.04 against an effect of 0.287.

3.7 Triangulation of three computational methods

Every trial computes its success probability three ways — exact binomial, Monte Carlo, and Chernoff bound where valid — on identical inputs. This is methodological triangulation, and each method plays a distinct role.

Exact binomial.
The ground truth. Every reported probability is this value; the other two exist to check it and to characterise their own usefulness.

Monte Carlo.
An independent numerical path to the same quantity, through a different code path (sampling rather than the survival function). Agreement is evidence that neither contains an error; its error against the exact value quantifies how much simulation is needed for a given precision (RQ2).

Chernoff bound.
A provable upper bound. Its role is not accuracy but *certification*: a margin the bound rules out is ruled out regardless of simulation effort. Its looseness is measured and reported rather than treated as a defect.

The three are computed on the same trial, not on separate runs, which is what makes the comparison a validation rather than a coincidence. Critically, the bound is applied only where mathematically valid: the dispatcher determines whether m lies in the upper tail, the lower tail, or neither, and returns no bound with a machine-readable reason outside those regions. Averaging the three quantities across heterogeneous trials — as an earlier iteration of this work did — compares them on different sub-populations, since the bound exists only where $m > \mu$; the detection-sweep figure of the *Findings* chapter therefore holds every other parameter fixed and sweeps only q.

3.8 Threshold solver design

The threshold solvers (T1–T7) invert the model, and three design decisions govern their trustworthiness.

Non-existence is a result. Each solver returns a structured object carrying either a value or an explicit absence with a machine-readable reason. No solver substitutes an endpoint, a zero or a sentinel for a threshold that does not exist, and the figures print "does not exist" rather than drawing a line. A threshold silently defaulted to a domain boundary would be indistinguishable, in a plot, from a real finding.

Exactness where the domain is discrete. T3, T6 and T7 search over integers and use binary search, which is exact because each underlying function is monotone in the search variable: $\mathbb{P}(X \geq m)$ is non-decreasing in n, and the Chernoff bound is decreasing in m above the mean. These are not approximations.

A definitional choice, stated. T4 and T5 required resolving an ambiguity in what "the attacker's utility" means. The headline optimiser searches $T \geq 0$ and therefore includes declining to attack, so its optimum is never negative — and a quantity floored at zero cannot be bisected for a zero crossing. The thresholds are consequently defined on $\max_{T \geq 1} U(T)$, the signed payoff of actually attacking, which goes negative exactly when attacking loses money. This is a definition rather than a derivation, it changes the reported numbers, and it is stated wherever those thresholds appear.

3.9 Coalition search design

The coalition analysis searches the budget simplex exhaustively at a 0.05 step (231 allocations for three vectors) rather than using continuous optimisation. The reason is structural: the objective is piecewise flat wherever $\lfloor B/c \rfloor$ does not change, so gradients are zero almost everywhere and undefined at the steps, which defeats gradient methods. Exhaustive evaluation is affordable because each evaluation is a convolution rather than a simulation. The reported optimum is the best grid point and may differ slightly from the true continuous optimum — stated as a limitation rather than glossed.

The success probability of a coalition is computed exactly. A sum of binomials with unequal success probabilities is not binomial and has no closed form, and the obvious approximations (normal, Poisson) are least accurate precisely in the tail that matters. Convolving the component PMFs is exact, and truncating each at $m-1$ before convolving is also exact rather than approximate, since $\mathbb{P}(W \geq m) = 1 - \mathbb{P}(W \leq m-1)$ and no combination summing to at most $m-1$ can involve a component larger than $m-1$. That truncation is what makes national-scale margins tractable: it holds the transform length at m rather than at the total attempt count, which is two orders of magnitude larger.

3.10 Validation and verification

Correctness is established at four levels.

Unit level (101 tests). Beyond routine coverage, several tests encode properties the mathematics requires and that a plausible-looking implementation can silently violate: that the lower-tail Chernoff bound carries a negative exponent, since a positive one yields a "bound" exceeding 1 that is vacuous but looks reasonable in output; that the bound is never returned outside its region of validity; that $T = 0$ rather than a division error when $B = 0$, $R = 0$ or $B < R$; and that each threshold solver's answer actually brackets the crossing it claims to find.

Cross-method level. The exact and Monte Carlo values are compared on every trial; the Chernoff bound is checked against the exact value on every trial where it is defined. A violation would appear as a bound lying below the exact value it is supposed to bound (Figure **??**); none occurs across 1,088, 3,471 and 3,604 valid trials respectively.

Cross-layer level. The abstract probabilities and the concrete mined-election flip rates are compared for ordering and magnitude (see the *Findings* chapter).

Reproducibility level. A test asserts that two runs at the same seed produce identical output, and a second asserts that different seeds do not, so the first cannot pass vacuously.

3.11 Reproducibility protocol

All randomness derives from a single seeded `numpy.random.Generator`; no global random state is used anywhere, and no unseeded standard-library random calls exist in the codebase. Re-running with seed 42 reproduces every number in this report bit-for-bit — verified by regenerating the full pipeline from a clean results directory and comparing the summary output. The threshold and coalition analyses are fully analytic and require no seed at all.

The report itself is part of the protocol. Every number in the prose is injected at build time from `results/json/` and `results/csv/` as a LaTeX macro; none is typed by hand. A macro whose experiment has not been run causes the document to fail compilation rather than to print a stale value, which makes it structurally impossible for the text to disagree with the data it describes.

3.12 Ethical considerations

This work models attacks on elections, which warrants explicit treatment.

No real system, election, voter or personal datum is involved at any point. All identifiers are simulation-internal indices; the candidates are A and B and have no referent. The artefact is a mathematical model, not a tool: it computes probabilities and contains no capability that could be directed at a deployed system, and nothing in it constitutes an exploit, a technique for evading a specific control, or an implementation of one.

The disclosure balance favours publication. The vectors modelled — bribery, Sybil registration, ballot stuffing — are long-established in the literature reviewed in Section 2, and the contribution is a quantification of their effect and of the countermeasure strength required to blunt them. That quantification is of substantially more use to a defender, who can act on a detection threshold, than to an attacker, who learns only that the attacks they already knew about work about as well as expected.

Table 3. Randomised parameter ranges. Every trial draws one value per symbol uniformly from these ranges; the protected configuration modifies only the acceptance and detection probabilities, never N, m, B, R, P, S or A.

Symbol	Meaning	Range
N	Electorate size	200 – 5,000
m	Manipulation threshold	2% – 25% of N
B	Attacker budget	1,000 – 200,000
R	Bribe reward per voter	10 – 500
P	Penalty if a bribed voter is caught	10 – 2,000
q	Bribery detection probability (baseline)	0.02 – 0.25
q	Bribery detection probability (protected)	baseline + 0.15 – 0.60
S	Attempted forged identities	10 – 1,000
q_s	Registration acceptance (baseline)	0.40 – 0.95
p_d	Sybil detection (baseline)	0.02 – 0.30
A	Attempted fraudulent ballots	10 – 1,000
q_b	Ballot acceptance (baseline)	0.30 – 0.90
p_b	Ballot detection (baseline)	0.02 – 0.30

Note. Source: `config.py::PARAM_RANGES`. Currency-like quantities are unitless in the model; only ratios such as B/R affect the results.

References

[1] B. Adida. Helios: Web-based Open-Audit Voting. In *Proceedings of the 17th USENIX Security Symposium*, pp. 335–348, 2008.

[2] G. S. Becker. Crime and Punishment: An Economic Approach. *Journal of Political Economy*, 76(2):169–217, 1968.

[3] J. Benaloh and D. Tuinstra. Receipt-Free Secret-Ballot Elections (Extended Abstract). In *Proceedings of the 26th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 544–553, 1994.

[4] M. Borge, E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly and B. Ford. Proof-of-Personhood: Redemocratizing Permissionless Cryptocurrencies. In *IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 23–26, 2017.

[5] H. Chernoff. A Measure of Asymptotic Efficiency for Tests of a Hypothesis Based on the Sum of Observations. *Annals of Mathematical Statistics*, 23(4):493–507, 1952.

[6] P. Daian, T. Kell, I. Bentov and A. Juels. On-Chain Vote Buying and the Rise of Dark DAOs. *Hacking, Distributed* (IC3 technical report), July 2018. Available at https://hackingdistributed.com/2018/07/02/on-chain-vote-buying/. *Cited as a technical report; not peer reviewed.*

[7] J. R. Douceur. The Sybil Attack. In *Proceedings of the 1st International Workshop on Peer-to-Peer Systems (IPTPS)*, LNCS 2429, pp. 251–260, 2002.

[8] I. Eyal and E. G. Sirer. Majority Is Not Enough: Bitcoin Mining Is Vulnerable. In *Financial Cryptography and Data Security (FC)*, LNCS 8437, pp. 436–454, 2014.

[9] P. Faliszewski, E. Hemaspaandra and L. A. Hemaspaandra. How Hard Is Bribery in Elections? *Journal of Artificial Intelligence Research*, 35:485–532, 2009.

[10] A. Juels, D. Catalano and M. Jakobsson. Coercion-Resistant Electronic Elections. In *Proceedings of the 2005 ACM Workshop on Privacy in the Electronic Society (WPES)*, pp. 61–70, 2005.

[11] P. Klimek, Y. Yegorov, R. Hanel and S. Thurner. Statistical Detection of Systematic Election Irregularities. *Proceedings of the National Academy of Sciences*, 109(41):16469–16473, 2012.

[12] Z. Liu, N. C. Luong, W. Wang, D. Niyato, P. Wang, Y.-C. Liang and D. I. Kim. A Survey on Applications of Game Theory in Blockchain. *arXiv preprint* arXiv:1902.10865, 2019.

[13] P. McCorry, S. F. Shahandashti and F. Hao. A Smart Contract for Boardroom Voting with Maximum Voter Privacy. In *Financial Cryptography and Data Security (FC)*, LNCS 10322, pp. 357–375, 2017.

[14] M. Mitzenmacher and E. Upfal. *Probability and Computing: Randomization and Probabilistic Techniques in Algorithms and Data Analysis*. 2nd ed., Cambridge University Press, 2017.

[15] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008.

[16] H. O. Ohize, A. J. Onumanyi, B. U. Umar, L. A. Ajao, R. O. Isah, E. M. Dogo, B. K. Nuhu, O. M. Olaniyi, J. G. Ambafi, V. B. Sheidu and M. M. Ibrahim. Blockchain for Securing Electronic Voting Systems: A Survey of Architectures, Trends, Solutions, and Challenges. *Cluster Computing*, 28, 2025. DOI 10.1007/s10586-024-04709-8.

[17] S. Park, M. Specter, N. Narula and R. L. Rivest. Going from Bad to Worse: From Internet Voting to Blockchain Voting. *Journal of Cybersecurity*, 7(1):tyaa025, 2021.

[18] Rahul, P. Gulia and N. S. Gill. Articulation of Blockchain Enabled E-Voting Systems: A Systematic Literature Review. *Peer-to-Peer Networking and Applications*, 18:142, 2025. DOI 10.1007/s12083-025-01956-3.

[19] M. A. Specter, J. Koppel and D. Weitzner. The Ballot is Busted Before the Blockchain: A Security Analysis of Voatz, the First Internet Voting Application Used in U.S. Federal Elections. In *Proceedings of the 29th USENIX Security Symposium*, pp. 1535–1553, 2020.

[20] D. Springall, T. Finkenauer, Z. Durumeric, J. Kitcat, H. Hursti, M. MacAlpine and J. A. Halderman. Security Analysis of the Estonian Internet Voting System. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 703–715, 2014.